The KISS
Klimt-Inspired Self-organizing System with Neural Cellular Automata

May 25, 2026

Michele Magrini

Abstract

We present *The KISS*, a Klimt-inspired self-organizing image generator based on Neural Cellular Automata (NCA). Starting from the framework of Growing Neural Cellular Automata (Mordvintsev et al., 2020), we train local neural update rules to reconstruct and persist an artistic 64×64 target inspired by Klimt's *The Kiss*. Unlike the original emoji-centered setting, our task requires preserving fine chromatic texture while maintaining the ability to grow from a single seed. We develop three models: a first grow model, an improved detail-preserving fine-tuned model, and a transition model that maps an already self-organized state to a zoomed target. A static web application allows interactive browser-side simulation of the trained models.

1. Introduction

Neural Cellular Automata are differentiable dynamical systems where each grid cell stores a vector state and repeatedly applies the same local neural rule. Global structure is therefore not drawn directly, but emerges from local communication. (Mordvintsev et al., 2020) showed that such systems can grow emoji-like images from a single active cell and can also learn persistence and regeneration. Related Distill work framed these systems as differentiable self-organizing processes and extended them toward texture synthesis (Niklasson et al., 2021).

This project keeps the core NCA idea but changes the target and training problem. Instead of simple icon-like targets, we reconstruct a Klimt-inspired image with stronger local color variation, requiring better detail retention and longer stability. The main challenge became balancing three competing objectives: growth from seed, detailed reconstruction, and persistence after many rollout steps. The final system is structured as a pipeline of specialized automata rather than a single model.

Email: Michele Magrini
<magrini.2066963@studenti.uniroma1.it>.

Deep Learning and Applied AI 2026, Sapienza University of Rome, 2nd semester a.y. 2025/2026.

2. Method

Each automaton state is a tensor $x \in \mathbb{R}^{H \times W \times C}$ with $H = W = 64$ and $C = 16$. The first four channels are interpreted as RGBA, while the remaining channels are hidden memory. The seed state is zero everywhere except at the central cell, where channels $3, \ldots, C-1$ are set to one, matching the convention used during training. At each step, each cell perceives local features through identity and derivative-like filters, including Sobel filters and, in later variants, a Laplacian-like detail cue. A shared 1×1 convolutional update network maps the perceived channels to a state increment. Updates are stochastic: with fire rate 0.5, only a random subset of cells is updated at each step, improving robustness to asynchronous dynamics. A living mask based on alpha prevents uncontrolled activation of dead regions.

Model 1: grow. Initially we train the initial model to grow *The Kiss* from the seed. We used $C = 16$, no target padding, batch size 8, pool size 1024 and fire rate 0.5. In each batch, samples are drawn from a pattern pool, ranked by current loss, and the first half is reset to the seed:

$$x_{0:4} \leftarrow x_{\text{seed}}.$$

The remaining samples come from the pool and train persistence. The rollout length is sampled as $T \sim \mathcal{U}(64, 256)$. The loss combines a mid-rollout and final objective:

$$\mathcal{L} = w_m \mathcal{L}_{pix}(x_{T/2}) + w_f \left(\mathcal{L}_{pix}(x_T) + w_d \mathcal{L}_{detail}(x_T) \right),$$

with $w_m = 0.2$, $w_f = 0.8$, $w_d = 0.15$. Pixel loss separates color and alpha:

$$\mathcal{L}_{pix} = 2.5 \| RGB - \widehat{RGB} \|^2 + 0.5 \| \alpha - \widehat{\alpha} \|^2.$$

This tuning was essential. With parameters close to the original emoji task, the model learned coarse shape but lacked detail. Increasing RGB importance and using wider

rollouts improved visual quality, but longer training caused a failure mode: the model became good at maintaining already formed patterns while forgetting how to grow from the seed. We therefore stopped at the last acceptable growing checkpoint, around 1200 training steps.

Model 2: improve. Then we fine-tune from the selected grow checkpoint. The old model is frozen and used as a teacher to generate input batches from the seed after 64–256 teacher steps. The student starts from these old-model states and is trained for longer rollouts, $T \sim \mathcal{U}(128, 512)$, with stronger final detail pressure: RGB weight 3.0, alpha weight 0.35, detail weight 0.20, and the same $0.2/0.8$ mid/final split. We also add standard L2 regularization (10^{-6}) and anchor regularization (10^{-5}) to keep the student close to the grow model. This prevents the new model from poisoning its own pool while improving texture and long-horizon stability.

Model 3: transition. At the end we train a new model from scratch, not a fine-tune. Its initial pool is generated by the improved old model rolled from the seed for 48–256 steps. The transition target is a second image, a zoomed version focused on the faces. During training, half of each batch can be refreshed with fresh old-model states, keeping the transition automaton tied to the distribution produced by Model 2. The transition model is rolled for 64–512 steps and optimized with RGB/alpha/detail losses plus L2 weight decay. Thus the pipeline becomes

$$\text{seed} \rightarrow \text{Model 2 full image} \rightarrow \text{Model 3 zoomed target}.$$

3. Results and Web Application

Qualitatively, the final pipeline shows three distinct behaviors: Model 1 grows a recognizable image from the seed; Model 2 improves detail while keeping growth; Model 3 learns a state-to-state visual transition. The most important empirical finding was that minimizing reconstruction loss alone is insufficient: models may obtain better pool loss while losing seed-growth ability. Monitoring seed rollouts and stopping before this collapse was crucial. The trained models are exposed in a static web application, available at https://mich1803.github.io/The-KISS/. The app runs the NCA simulation in the browser, allows model selection between weak and good checkpoints, and includes play, pause, restart, speed control, and a zoom transition.

4. Conclusion

The KISS demonstrates that NCAs can be adapted from simple morphogenetic image growth to a more demanding artistic reconstruction and transition pipeline. The main contribution is not a new architecture, but a practical training strategy: separate grow, improve, and transition phases; preserve seed rollouts; increase RGB and detail losses; and avoid relying only on self-generated persistent pools. Future work should add quantitative image metrics, ablations, and more systematic robustness tests.

Code and Reproducibility

The full project repository is available at https://github.com/mich1803/The-KISS. It contains the Colab training notebooks, exported checkpoint models, and the static web application used to simulate the trained Neural Cellular Automata.

Statement on the use of AI

AI-based tools were used to support the project in the following ways: initially finding bibliography and related work; refactoring some core code functions; suggesting ideas to improve the project, including losses and hyperparameter tuning; summarizing original papers; rephrasing and shortening this report; and building the full static web application. All code, experiments, results, and final claims were reviewed and remain the author's responsibility.

References

Mordvintsev, A., Randazzo, E., Niklasson, E., and Levin, M. Growing neural cellular automata. *Distill*, 5(2):e23, 2020. doi: 10.23915/distill.00023. URL https://distill.pub/2020/growing-ca.

Niklasson, E., Mordvintsev, A., Randazzo, E., and Levin, M. Self-organising textures. *Distill*, 2021. doi: 10.23915/distill.00027.003. URL https://distill.pub/selforg/2021/textures.

Appendix

The following pages provide additional training visualizations. For each model, we report the generated outputs at selected epochs together with the corresponding loss curve, showing how the visual reconstruction and optimization process evolve during training.

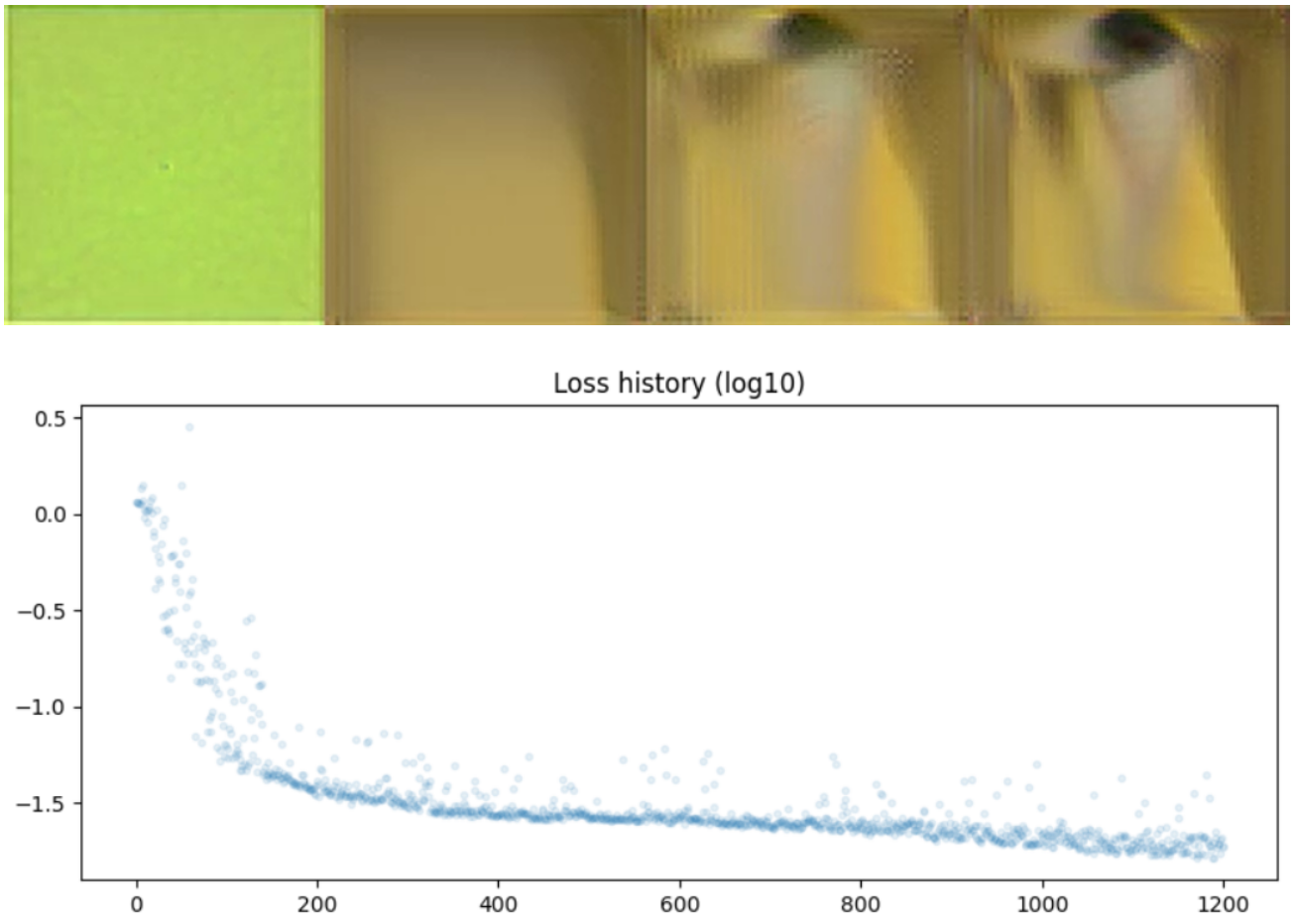
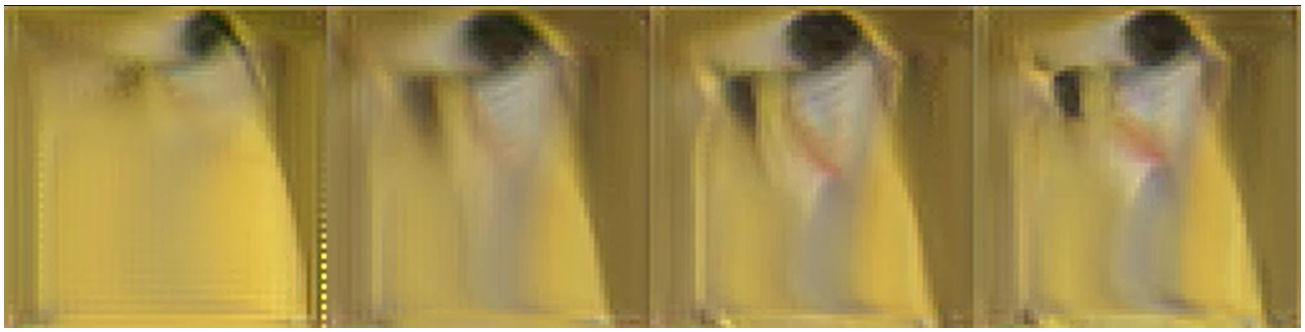


Figure 1. Model 1 (Grow): qualitative results at different epochs and corresponding training loss.



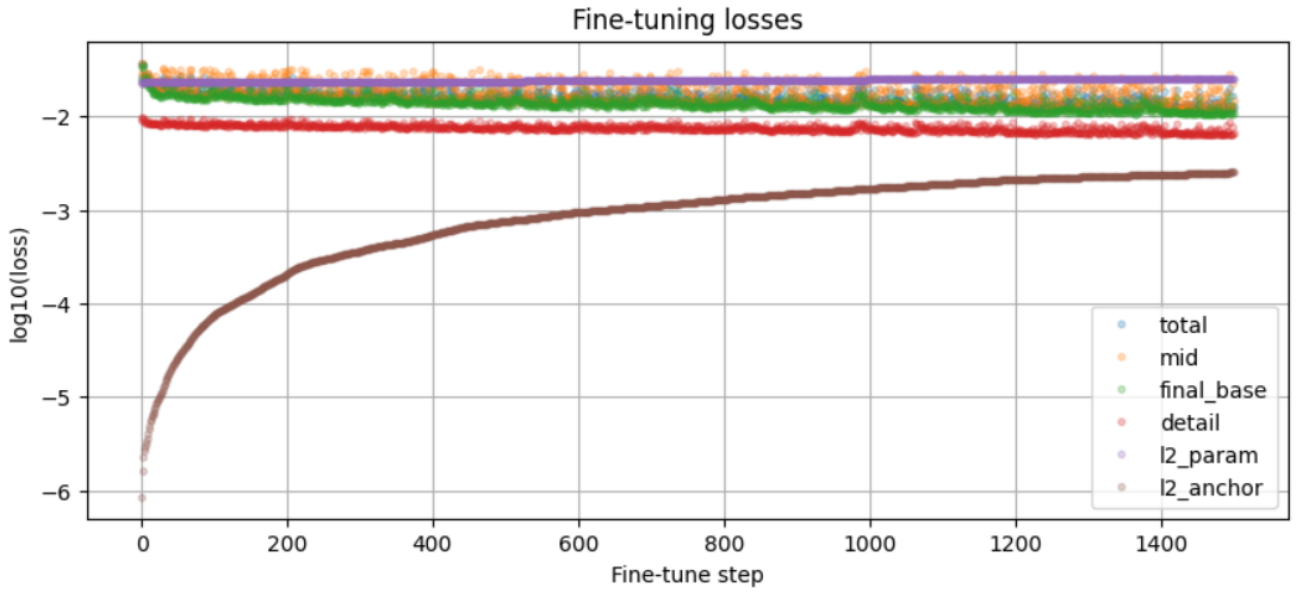


Figure 2. Model 2 (Improve): fine-tuning progression and loss curve.

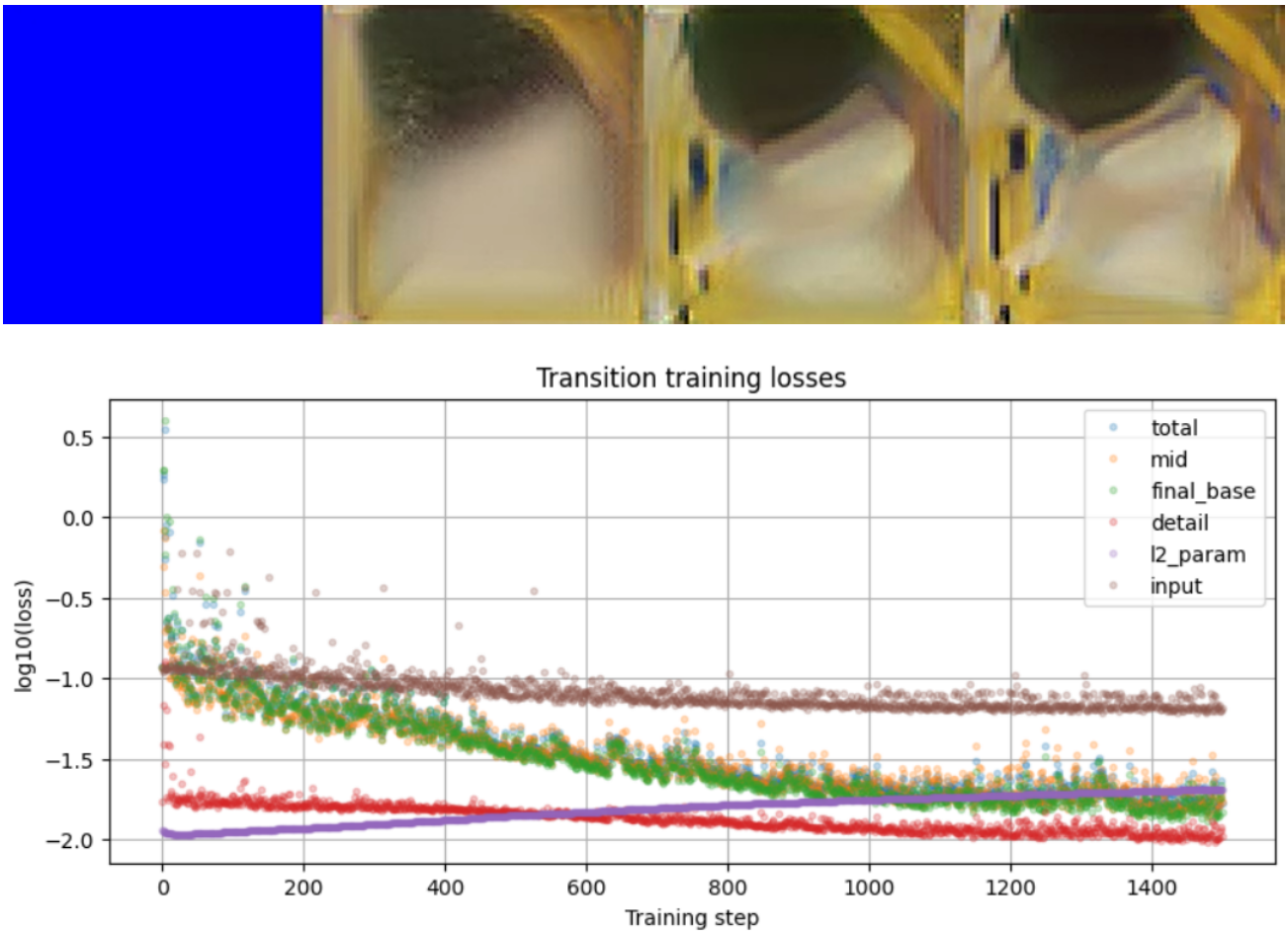


Figure 3. Model 3 (Transition): transition results at different epochs and training loss.